Data Preparation and Exploratory Data Analysis

```
# Import necessary libraries and set visualization style
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# Visualization style
sns.set(style="whitegrid")
plt.rcParams["figure.figsize"] = (10,6)
```

Double-click (or enter) to edit

Double-click (or enter) to edit

```
# Mount Google Drive to access datasets
from google.colab import drive
drive.mount('/content/drive')
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount, call drive.mount("/content/drive", forc
```

```
# Load MovieLens datasets (ratings and movies) from Google Drive
ratings_file_path = '/content/drive/MyDrive/DataSetMovieLens/ratings.csv'
movies_file_path = '/content/drive/MyDrive/DataSetMovieLens/movies.csv'
tags_file_path = '/content/drive/MyDrive/DataSetMovieLens/tags.csv'
# Load the datasets
ratings = pd.read_csv(ratings_file_path)
movies = pd.read_csv(movies_file_path)
tags = pd.read_csv(tags_file_path)

# Preview data
print("Ratings Dataset")
display(ratings.head())

print("\nMovies Dataset")
display(movies.head())

print("\nTags Dataset")
display(tags.head())
```

```
Ratings Dataset
```

	userId	movieId	rating	timestamp
0	1	1	4.0	964982703
1	1	3	4.0	964981247
2	1	6	4.0	964982224
3	1	47	5.0	964983815
4	1	50	5.0	964982931

```
Movies Dataset
```

	movieId	title	genres
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy
1	2	Jumanji (1995)	Adventure|Children|Fantasy
2	3	Grumpier Old Men (1995)	Comedy|Romance
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance
4	5	Father of the Bride Part II (1995)	Comedy

```
Tags Dataset
```

	userId	movieId	tag	timestamp
0	2	60756	funny	1445714994
1	2	60756	Highly quotable	1445714996
2	2	60756	will ferrell	1445714992
3	2	89774	Boxing story	1445715207
4	2	89774	MMA	1445715200

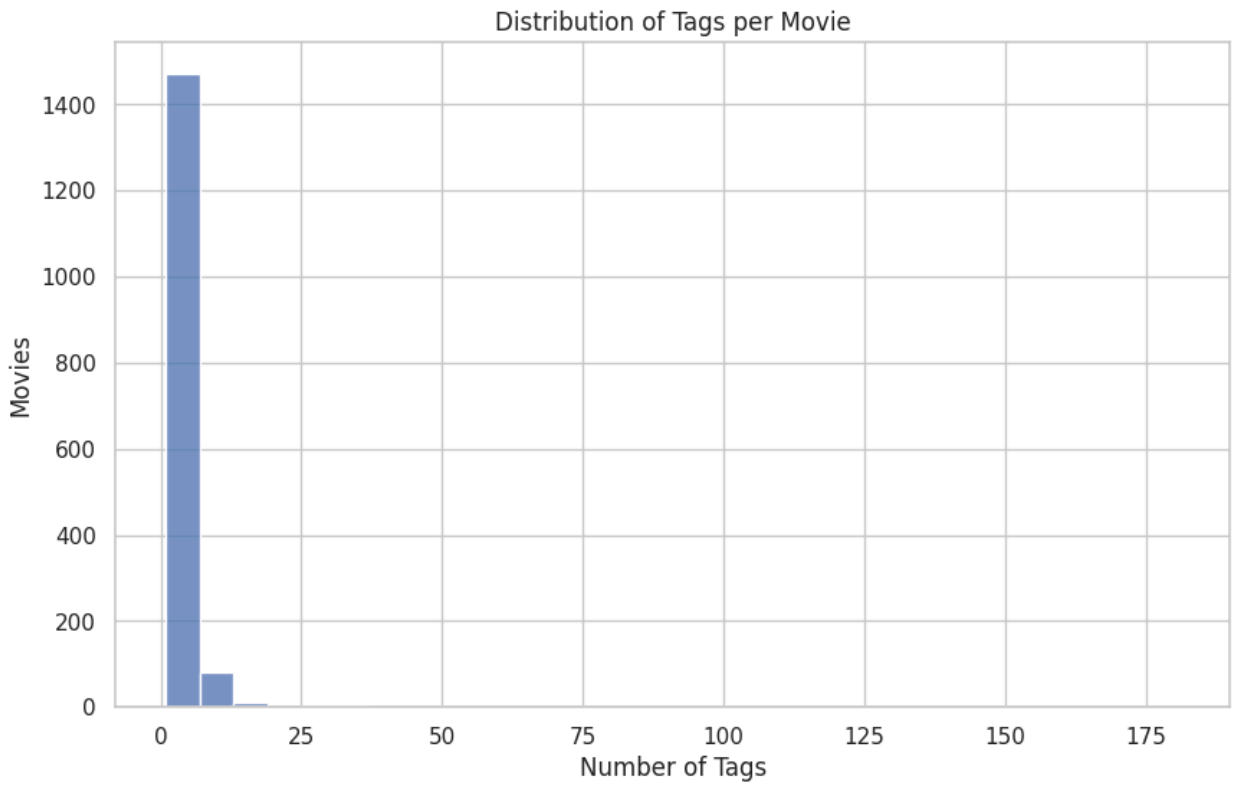
```
# Get initial overview of data shapes and types for both datasets
print("Ratings Shape:", ratings.shape)
print("Movies Shape:", movies.shape)

print("\nRatings Info")
ratings.info()

print("\nMovies Info")
movies.info()
```

```
Ratings Shape: (100836, 4)
Movies Shape: (9742, 3)

Ratings Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100836 entries, 0 to 100835
Data columns (total 4 columns):
 #   Column     Non-Null Count   Dtype
---  ------     --------------   -----
 0   userId     100836 non-null  int64
 1   movieId    100836 non-null  int64
 2   rating     100836 non-null  float64
 3   timestamp  100836 non-null  int64
dtypes: float64(1), int64(3)
memory usage: 3.1 MB

Movies Info
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9742 entries, 0 to 9741
Data columns (total 3 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   movieId  9742 non-null   int64
 1   title    9742 non-null   object
 2   genres   9742 non-null   object
dtypes: int64(1), object(2)
memory usage: 228.5+ KB
```

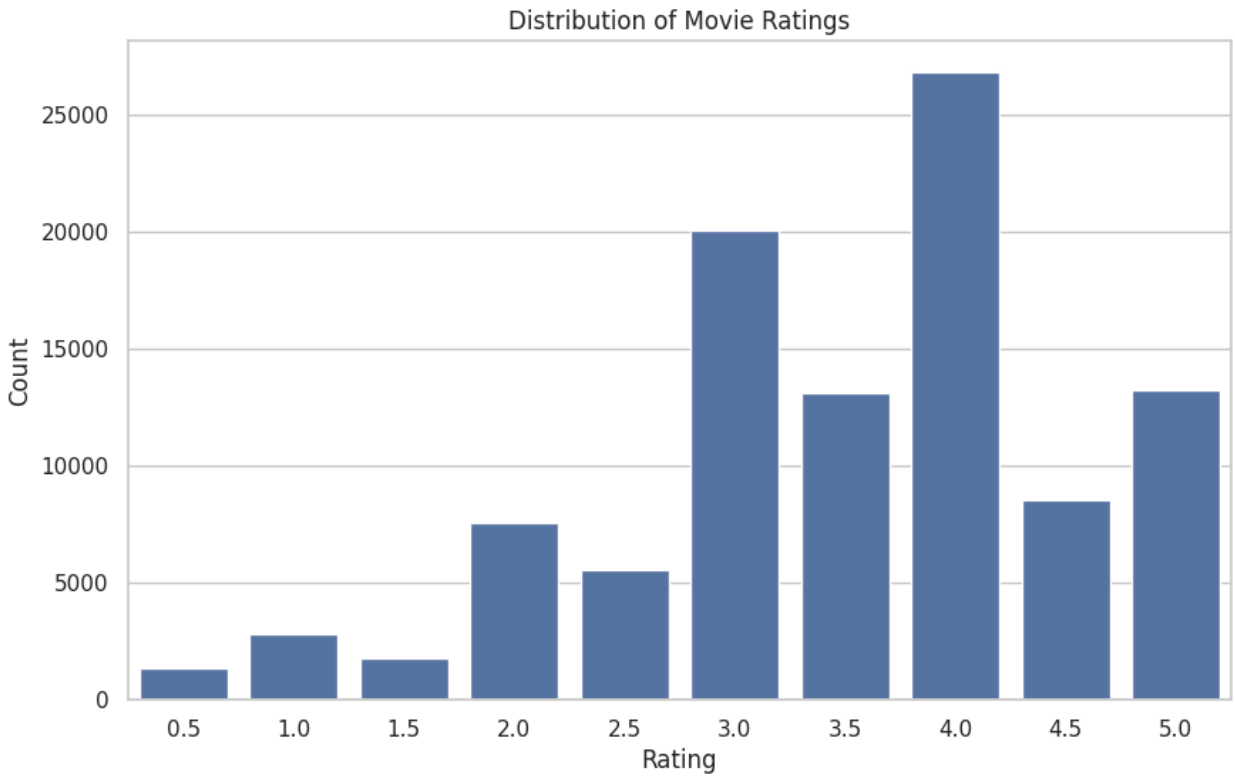
```
# Calculate and display the number of unique users and movies
num_users = ratings['userId'].nunique()
num_movies = ratings['movieId'].nunique()

print("Number of unique users:", num_users)
print("Number of unique movies:", num_movies)
```

```
Number of unique users: 610
Number of unique movies: 9724
```

```
tags_per_movie = tags.groupby('movieId').size()

plt.figure()
sns.histplot(tags_per_movie, bins=30)
plt.title("Distribution of Tags per Movie")
plt.xlabel("Number of Tags")
plt.ylabel("Movies")
plt.show()
```



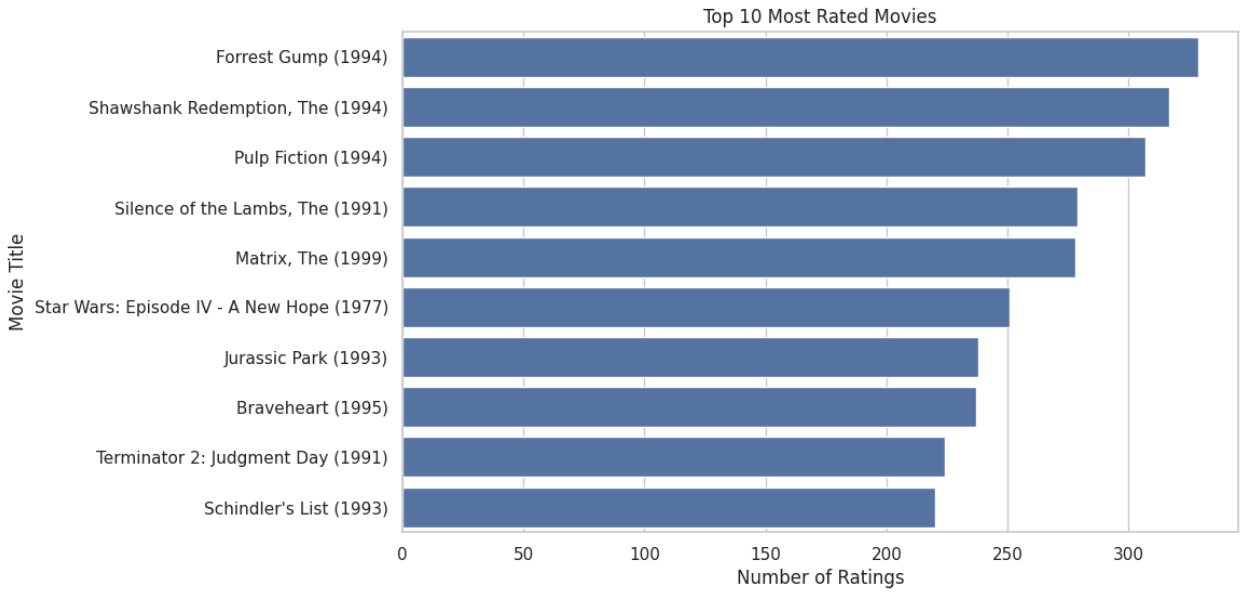

```
# Visualize the distribution of movie ratings
plt.figure()
sns.countplot(x="rating", data=ratings)
plt.title("Distribution of Movie Ratings")
plt.xlabel("Rating")
plt.ylabel("Count")
plt.show()
```




```
# Identify and visualize the top 10 most rated movies
movie_ratings_count = ratings.groupby('movieId').size().sort_values(ascending=False)

top_movies = movie_ratings_count.head(10)

top_movies = top_movies.reset_index()
top_movies.columns = ['movieId', 'rating_count']

# Merge with movie titles
top_movies = top_movies.merge(movies, on='movieId')

print("Top 10 Most Rated Movies")
display(top_movies[['title', 'rating_count']])

# Plot
plt.figure()
sns.barplot(y='title', x='rating_count', data=top_movies)
plt.title("Top 10 Most Rated Movies")
plt.xlabel("Number of Ratings")
plt.ylabel("Movie Title")
plt.show()
```

```
Top 10 Most Rated Movies
```

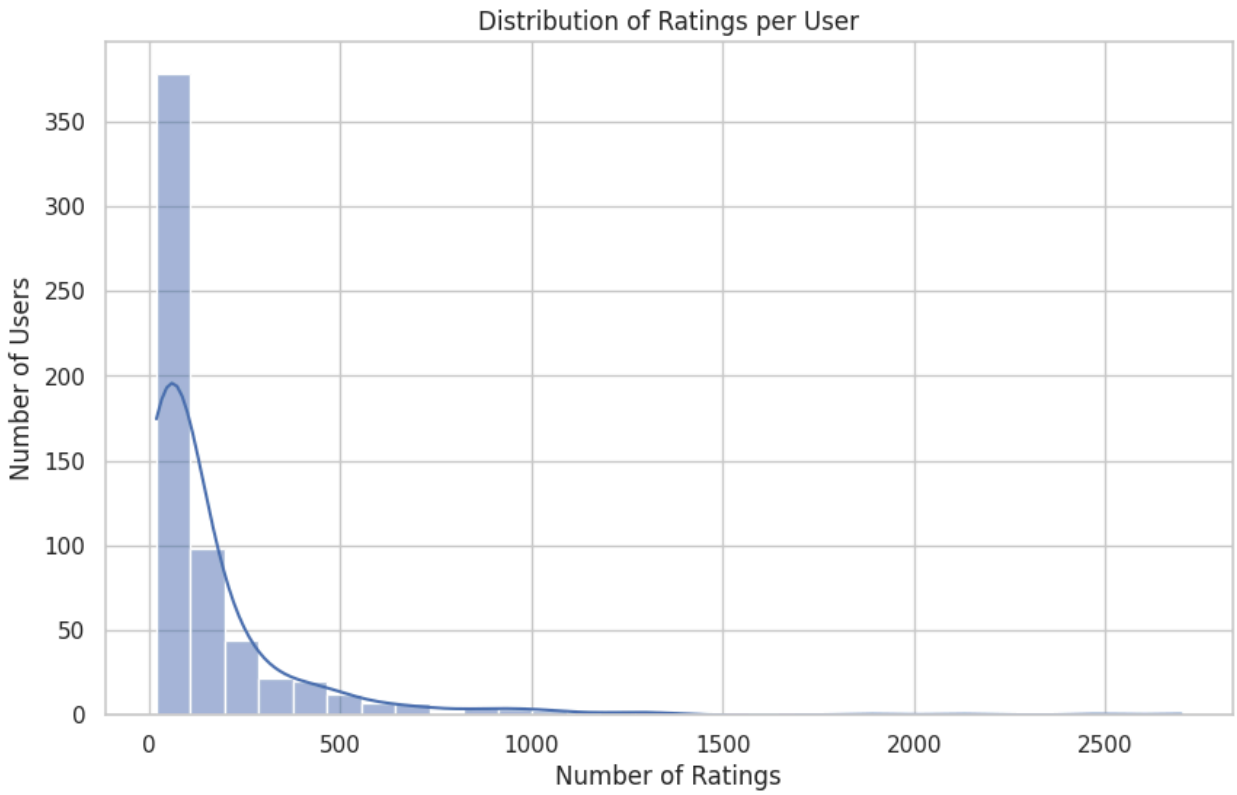
	title	rating_count
0	Forrest Gump (1994)	329
1	Shawshank Redemption, The (1994)	317
2	Pulp Fiction (1994)	307
3	Silence of the Lambs, The (1991)	279
4	Matrix, The (1999)	278
5	Star Wars: Episode IV - A New Hope (1977)	251
6	Jurassic Park (1993)	238
7	Braveheart (1995)	237
8	Terminator 2: Judgment Day (1991)	224
9	Schindler's List (1993)	220



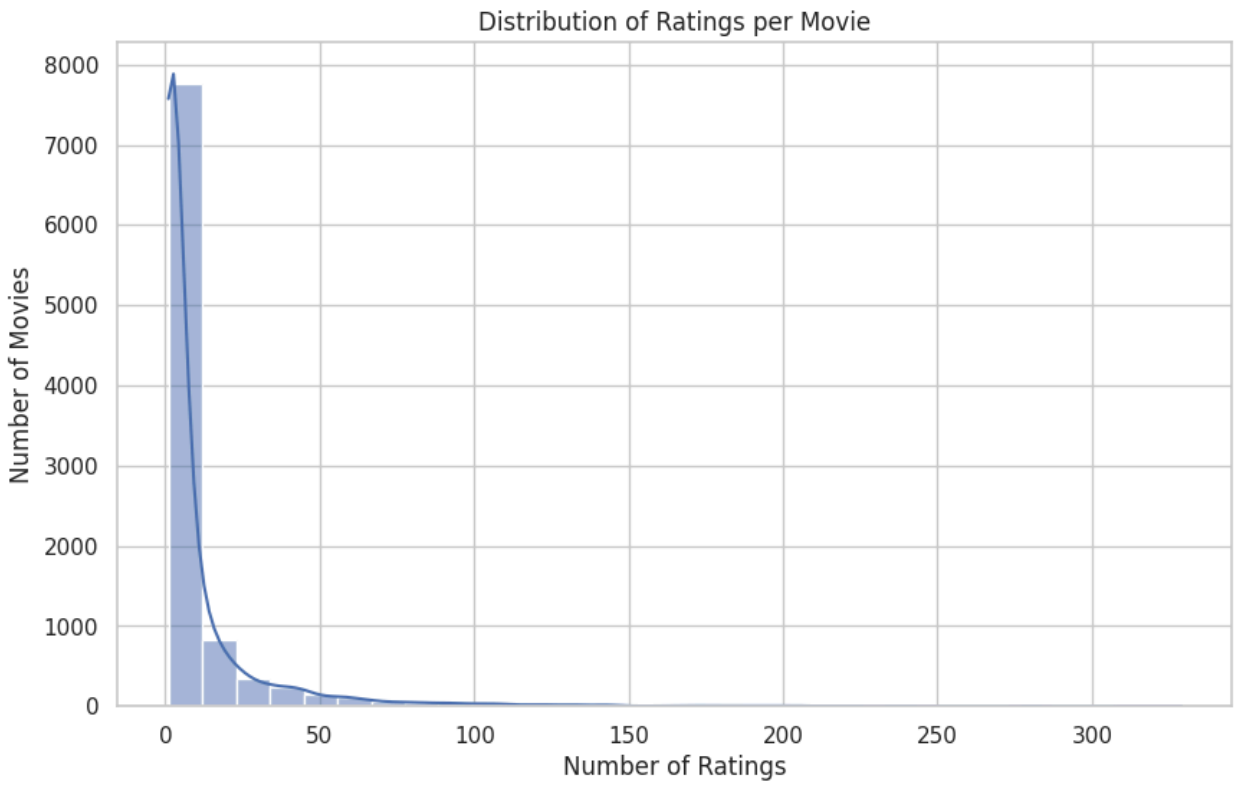

```
# Calculate and display the sparsity of the user-item matrix
user_item_matrix = ratings.pivot(index='userId', columns='movieId', values='rating')

num_possible_ratings = user_item_matrix.shape[0] * user_item_matrix.shape[1]
num_actual_ratings = ratings.shape[0]

sparsity = (1 - (num_actual_ratings / num_possible_ratings)) * 100

print("User-Item Matrix Shape:", user_item_matrix.shape)
print("Matrix Sparsity: {:.2f}%".format(sparsity))
```

```
User-Item Matrix Shape: (610, 9724)
Matrix Sparsity: 98.30%
```

```
# Visualize the distribution of ratings per user
ratings_per_user = ratings.groupby('userId').size()

plt.figure()
sns.histplot(ratings_per_user, bins=30, kde=True)
plt.title("Distribution of Ratings per User")
plt.xlabel("Number of Ratings")
plt.ylabel("Number of Users")
plt.show()
```




```
# Visualize the distribution of ratings per movie
ratings_per_movie = ratings.groupby('movieId').size()

plt.figure()
sns.histplot(ratings_per_movie, bins=30, kde=True)
plt.title("Distribution of Ratings per Movie")
plt.xlabel("Number of Ratings")
plt.ylabel("Number of Movies")
plt.show()
```



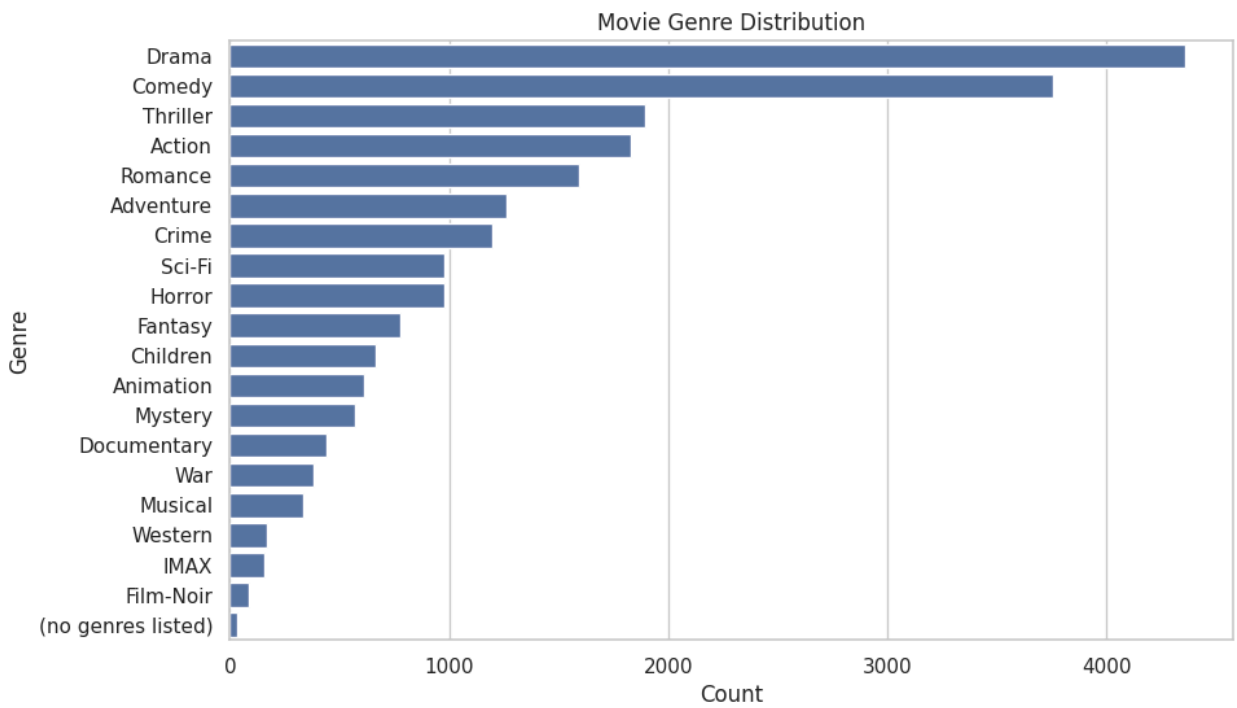

```
# Identify and display the top 10 highest-rated movies
movie_avg_rating = ratings.groupby('movieId')['rating'].mean()
```

```
movie_avg_rating = movie_avg_rating.reset_index()
movie_avg_rating = movie_avg_rating.merge(movies, on='movieId')

top_rated = movie_avg_rating.sort_values(by='rating', ascending=False).head(10)

print("Top 10 Highest Rated Movies")
display(top_rated[['title','rating']])
```

```
Top 10 Highest Rated Movies
```

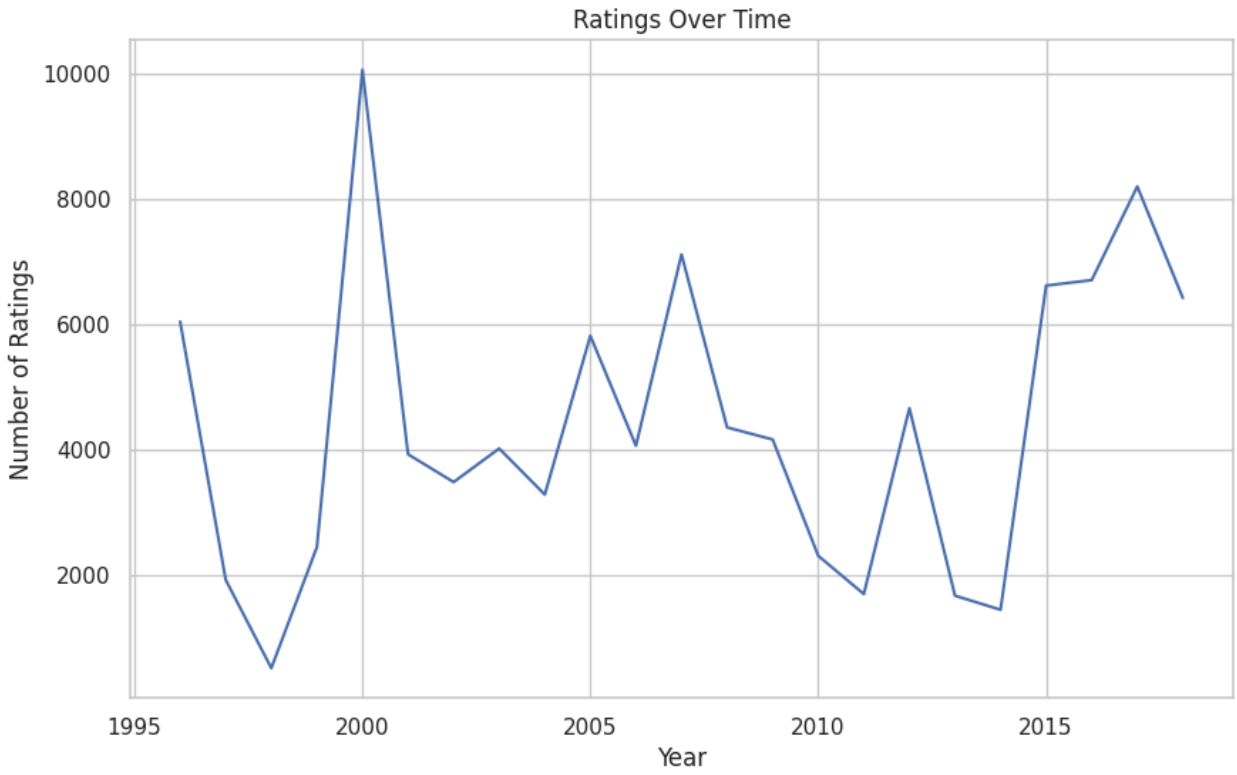
	title	rating
7638	Paper Birds (Pájaros de papel) (2010)	5.0
8089	Act of Killing, The (2012)	5.0
9065	Jump In! (2007)	5.0
9076	Human (2015)	5.0
9078	L.A. Slasher (2015)	5.0
4245	Lady Jane (1986)	5.0
8136	Bill Hicks: Revelations (1993)	5.0
8130	Justice League: Doom (2012)	5.0
4240	Open Hearts (Elsker dig for evigt) (2002)	5.0
9104	Formula of Love (1984)	5.0

```
# Explore and visualize the distribution of movie genres
genres = movies['genres'].str.split('|').explode()

plt.figure()
sns.countplot(y=genres, order=genres.value_counts().index)
plt.title("Movie Genre Distribution")
plt.xlabel("Count")
plt.ylabel("Genre")
plt.show()
```




```
# Analyze and visualize the trend of ratings over time (by year)
ratings['timestamp'] = pd.to_datetime(ratings['timestamp'], unit='s')

ratings['year'] = ratings['timestamp'].dt.year

ratings_per_year = ratings.groupby('year').size()
```

```
plt.figure()
ratings_per_year.plot()
plt.title("Ratings Over Time")
plt.xlabel("Year")
plt.ylabel("Number of Ratings")
plt.show()
```



Content-Based Recommender with Metadata Features

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

#preproces Genres
#replaced '|' with spaces so TF-IDF can treat them as words

movies['genres_processed'] = movies['genres'].str.replace('|', ' ', regex=False)
# Preview
movies[['title', 'genres', 'genres_processed']].head()
```

	title	genres	genres_processed
0	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	Adventure Animation Children Comedy Fantasy
1	Jumanji (1995)	Adventure|Children|Fantasy	Adventure Children Fantasy
2	Grumpier Old Men (1995)	Comedy|Romance	Comedy Romance
3	Waiting to Exhale (1995)	Comedy|Drama|Romance	Comedy Drama Romance

```
#Aggregate Tags for Each Movie

# Convert tags to string (avoid NaN issues)
tags['tag'] = tags['tag'].astype(str)

# Aggregate all tags belonging to the same movie
movie_tags = tags.groupby('movieId')['tag'].apply(lambda x: " ".join(x)).reset_index()

movie_tags.head()
```

	movieId	tag
0	1	pixar pixar fun
1	2	fantasy magic board game Robin Williams game
2	3	moldy old
3	5	pregnancy remake
4	7	remake

Next steps: Generate code with movie_tags New interactive sheet

```
#Merge Tags with Movies

movies_with_tags = movies.merge(movie_tags, on='movieId', how='left')

# Replace missing tags with empty string
movies_with_tags['tag'] = movies_with_tags['tag'].fillna('')

movies_with_tags.head()
```

	movieId	title	genres	genres_processed	tag
0	1	Toy Story (1995)	Adventure|Animation|Children|Comedy|Fantasy	Adventure Animation Children Comedy Fantasy	pixar pixar fun
1	2	Jumanji (1995)	Adventure|Children|Fantasy	Adventure Children Fantasy	fantasy magic board game Robin Williams game
2	3	Grumpier Old Men (1995)	Comedy|Romance	Comedy Romance	moldy old
3	4	Waiting to Exhale (1995)	Comedy|Drama|Romance	Comedy Drama Romance	
4	5	Father of the Bride Part II (1995)	Comedy	Comedy	pregnancy remake

Next steps: Generate code with movies_with_tags New interactive sheet

```
#Create Combined Feature (Genres + Tags)

movies_with_tags['combined_features'] = (
    movies_with_tags['genres_processed'] + " " + movies_with_tags['tag']
)

movies_with_tags[['title','combined_features']].head()
```

	title	combined_features
0	Toy Story (1995)	Adventure Animation Children Comedy Fantasy pi...
1	Jumanji (1995)	Adventure Children Fantasy fantasy magic board...
2	Grumpier Old Men (1995)	Comedy Romance moldy old
3	Waiting to Exhale (1995)	Comedy Drama Romance
4	Father of the Bride Part II (1995)	Comedy pregnancy remake

```
#TF-IDF Vectorization

tfidf = TfidfVectorizer(stop_words='english')
tfidf_matrix = tfidf.fit_transform(movies_with_tags['combined_features'])
print("TF-IDF Matrix Shape:", tfidf_matrix.shape)
```

```
TF-IDF Matrix Shape: (9742, 1677)
```

```
# Cosine Similarity
```

```
cosine_sim = cosine_similarity(tfidf_matrix, tfidf_matrix)
print("Cosine Similarity Matrix Shape:", cosine_sim.shape)
```

```
Cosine Similarity Matrix Shape: (9742, 9742)
```

```
# Building Movie Index Mapping

indices = pd.Series(movies_with_tags.index, index=movies_with_tags['title']).drop_duplicates()
indices.head()
```

title	0
Toy Story (1995)	0
Jumanji (1995)	1
Grumpier Old Men (1995)	2
Waiting to Exhale (1995)	3
Father of the Bride Part II (1995)	4

dtype: int64

```
# 8. Recommendation Function
def recommend_movies(title, cosine_sim=cosine_sim):

    # Get movie index
    idx = indices.get(title)

    if idx is None:
        print("Movie not found in dataset")
        return

    # Get similarity scores
    sim_scores = list(enumerate(cosine_sim[idx]))

    # Sort movies by similarity
    sim_scores = sorted(sim_scores, key=lambda x: x[1], reverse=True)

    # Remove the movie itself
    sim_scores = sim_scores[1:11]

    # Get movie indices
    movie_indices = [i[0] for i in sim_scores]

    # Return movie titles
    return movies_with_tags['title'].iloc[movie_indices]
```

```
import re
import ipywidgets as widgets
from IPython.display import display, clear_output

#word based search function
def search_movies(query):
    pattern = re.compile(r'\b' + re.escape(query.lower()))

    matches = [
        title for title in movies_with_tags['title']
        if pattern.search(title.lower())
    ]

    return matches[:20]

# UI widgets
search_box = widgets.Text(
    placeholder='Type a movie name (e.g. Toy)',
    description='Search:',
    layout=widgets.Layout(width='50%')
)

results = widgets.Select(
    options=[],
```

```
    rows=10,
    description='Movies:',
    layout=widgets.Layout(width='50%')
)

output = widgets.Output()

# Update results while typing
def update_results(change):
    query = change['new']

    if len(query) >= 2:
        results.options = search_movies(query)
    else:
        results.options = []

search_box.observe(update_results, names='value')

# Show recommendations
def show_recommendations(change):
    movie = change['new']
    with output:
        clear_output()

        if movie not in indices:
            print("Movie not found.")
            return

        print(f"\nTop 10 recommendations for: {movie}\n")

        recs = recommend_movies(movie)

        for i, r in enumerate(recs, 1):
            print(f"{i}. {r}")

results.observe(show_recommendations, names='value')
display(search_box)
display(results)
display(output)
```

Search: Type a movie name (e.g. Toy)

Movies:

User-User Collaborative Filtering

```
#creating User Item-Matrix
# Rows = users
# Columns = movies
# Values = ratings

user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
)

# Fill missing ratings with 0 (needed for cosine similarity)
user_item_filled = user_item_matrix.fillna(0)
user_item_filled.head(5)
```

movieId	1	2	3	4	5	6	7	8	9	10	...	193565	193567	193571	193573	193579	193581	193583
userId																		
1	4.0	0.0	4.0	0.0	0.0	4.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
2	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0
5	4.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0	0.0	0.0	0.0	0.0

5 rows × 9724 columns

```
#Computing Cosine Similarity
# Cosine similarity between user vectors

user_vectors = user_item_filled.values

# Compute cosine similarity manually using numpy
dot_product = np.dot(user_vectors, user_vectors.T)

norms = np.linalg.norm(user_vectors, axis=1)

user_similarity = dot_product / (norms[:, None] * norms[None, :])

# Convert to DataFrame for easier indexing
user_similarity_df = pd.DataFrame(
    user_similarity,
    index=user_item_matrix.index,
    columns=user_item_matrix.index
)
user_similarity_df.head(5)
```

userId	1	2	3	4	5	6	7	8	9	10	...	
userId												
1	1.000000	0.027283	0.059720	0.194395	0.129080	0.128152	0.158744	0.136968	0.064263	0.016875	...	0.080
2	0.027283	1.000000	0.000000	0.003726	0.016614	0.025333	0.027585	0.027257	0.000000	0.067445	...	0.202
3	0.059720	0.000000	1.000000	0.002251	0.005020	0.003936	0.000000	0.004941	0.000000	0.000000	...	0.005
4	0.194395	0.003726	0.002251	1.000000	0.128659	0.088491	0.115120	0.062969	0.011361	0.031163	...	0.085
5	0.129080	0.016614	0.005020	0.128659	1.000000	0.300349	0.108342	0.429075	0.000000	0.030611	...	0.068

5 rows × 610 columns

```
# Predict Ratings using User-User CF
def predict_ratings(user_id):
    sim_scores = user_similarity_df.loc[user_id]
    weighted_ratings = np.dot(sim_scores, user_item_filled)
    sim_sum = np.abs(sim_scores).sum()

    if sim_sum == 0:
        predicted = np.zeros(len(user_item_matrix.columns))
    else:
        predicted = weighted_ratings / sim_sum

    return pd.Series(predicted, index=user_item_matrix.columns)


# Recommendation function
def recommend_movies_user(user_id, top_n=10):
    user_predictions = predict_ratings(user_id)
    user_predictions = user_predictions.sort_values(ascending=False)

    top_movie_ids = user_predictions.head(top_n).index

    recommendations = movies[movies['movieId'].isin(top_movie_ids)].copy()
    recommendations = recommendations[['movieId', 'title']]
```

```
    return recommendations
```

```
# #Recommendation FUnction
# def recommend_movies_user(user_id, top_n=10):

#       user_predictions = predict_ratings(user_id)
#       user_predictions = user_predictions.sort_values(ascending=False)
#       top_movie_ids = user_predictions.head(top_n).index
#       recommendations = movies[movies['movieId'].isin(top_movie_ids)]
#       return recommendations[['title']]
```

```
import ipywidgets as widgets
from IPython.display import display, clear_output

user_box = widgets.IntText(
    value=1,
    description='User ID:'
)

button = widgets.Button(description="Recommend Movies")

output = widgets.Output()

def recommend_for_user(b):

    with output:
        clear_output()

        user_id = user_box.value

        print(f"\nTop recommendations for User {user_id}:\n")

        recs = recommend_movies_user(user_id)

        for i, title in enumerate(recs['title'], 1):
            print(f"{i}. {title}")

button.on_click(recommend_for_user)

display(user_box, button, output)
```

User ID: 1

Recommend Movies

```
Top recommendations for User 1:

1. Braveheart (1995)
2. Star Wars: Episode IV - A New Hope (1977)
3. Pulp Fiction (1994)
4. Shawshank Redemption, The (1994)
5. Forrest Gump (1994)
6. Jurassic Park (1993)
7. Terminator 2: Judgment Day (1991)
8. Silence of the Lambs, The (1991)
9. Star Wars: Episode V - The Empire Strikes Back (1980)
10. Matrix, The (1999)
```

Matrix Factorisation using SVD

```
#Create User-Item Matrix\

user_item_matrix = ratings.pivot_table(
    index='userId',
    columns='movieId',
    values='rating'
)
# Fill missing values with 0
R = user_item_matrix.fillna(0).values
```

```
#Apply Singular Value Decomposition
# Decompose matrix into latent factors

U, sigma, Vt = np.linalg.svd(R, full_matrices=False)
# Convert sigma to diagonal matrix
sigma = np.diag(sigma)
```

```
#Reduce dimensionality
# Keep only top k features

k = 50
U_k = U[:, :k]
sigma_k = sigma[:k, :k]
Vt_k = Vt[:k, :]
```

```
#Reconstruct predicted rating matrix
# Approximate the original matrix

R_pred = np.dot(np.dot(U_k, sigma_k), Vt_k)

predicted_ratings = pd.DataFrame(
    R_pred,
    index=user_item_matrix.index,
    columns=user_item_matrix.columns
)
```

```
# Recommend movies for a user using SVD
def recommend_movies_svd(user_id, top_n=10):

    user_predictions = predicted_ratings.loc[user_id].copy()
    top_movie_ids = user_predictions.sort_values(ascending=False).head(top_n).index

    recommendations = movies[movies['movieId'].isin(top_movie_ids)].copy()
    recommendations = recommendations[['movieId', 'title']]

    return recommendations

import ipywidgets as widgets
from IPython.display import display, clear_output


user_box = widgets.IntText(
    value=1,
    description='User ID:'
)

button = widgets.Button(
    description="Recommend Movies",
    button_style='success'
)

output = widgets.Output()


def recommend_for_user(b):

    with output:
        clear_output()

        user_id = user_box.value

        if user_id not in user_item_matrix.index:
            print("User ID not found.")
            return

        print(f"\nTop recommendations for User {user_id} (Matrix Factorization):\n")

        recs = recommend_movies_svd(user_id)

        for i, title in enumerate(recs['title'], 1):
            print(f"{i}. {title}")
```

```
button.on_click(recommend_for_user)
display(user_box, button, output)
```

User ID: 1

Recommend Movies

```
Top recommendations for User 1:

1. Braveheart (1995)
2. Star Wars: Episode IV - A New Hope (1977)
3. Pulp Fiction (1994)
4. Shawshank Redemption, The (1994)
5. Forrest Gump (1994)
6. Jurassic Park (1993)
7. Terminator 2: Judgment Day (1991)
8. Silence of the Lambs, The (1991)
9. Star Wars: Episode V - The Empire Strikes Back (1980)
10. Matrix, The (1999)
```

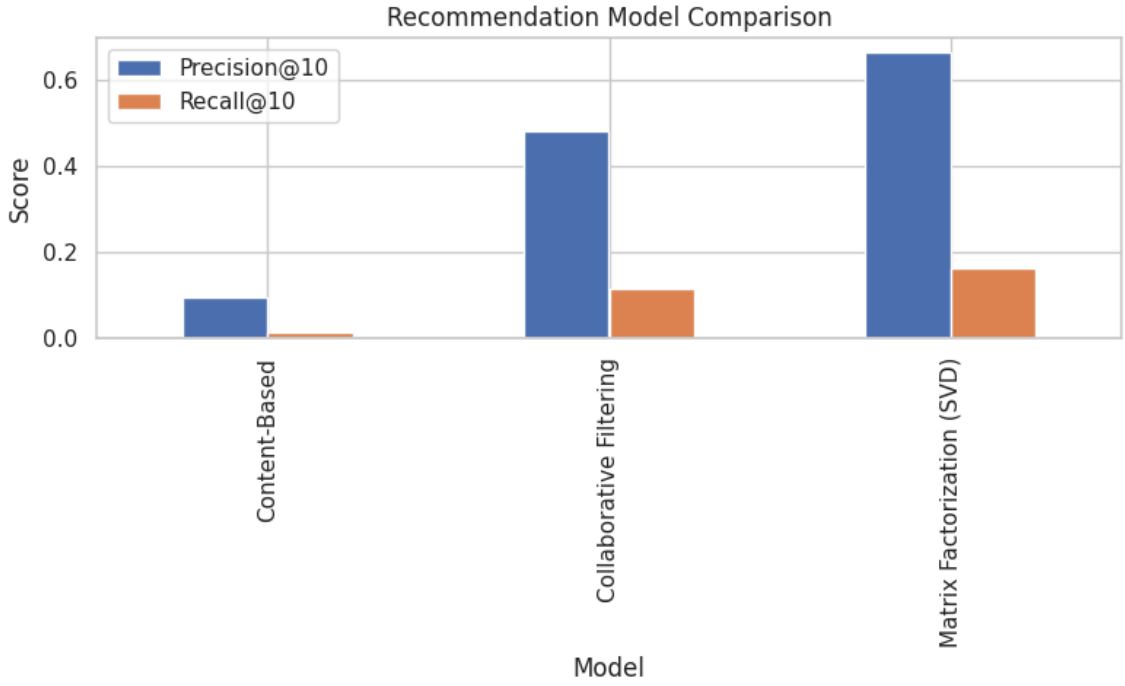
Model Evaluation and Comparison

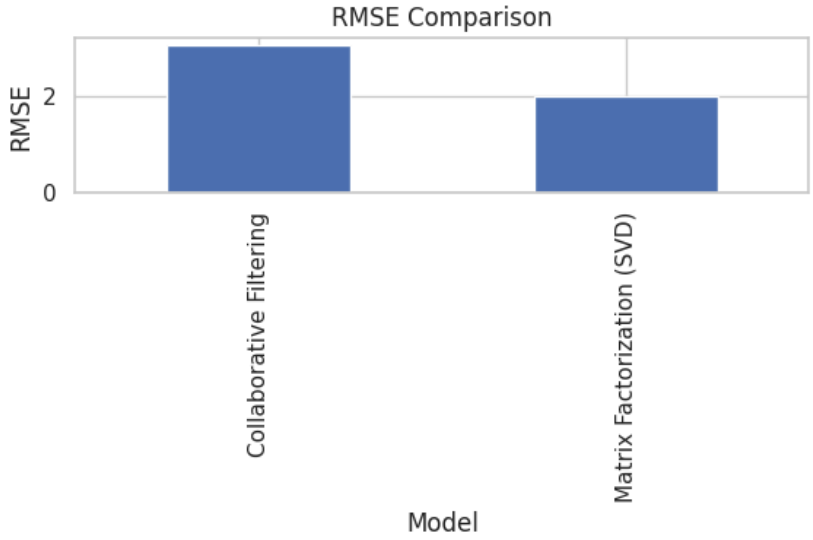
```
# ========================================
# Model Evaluation (Fixed)
# ========================================

RELEVANCE_THRESHOLD = 4
K = 10
users = ratings['userId'].unique()[:50]

precision_content, recall_content = [], []
precision_cf, recall_cf = [], []
precision_svd, recall_svd = [], []


# Precision@K
def precision_at_k(recommended, relevant, k=10):
    recommended = recommended[:k]
    hits = len(set(recommended) & set(relevant))
    return hits / k


# Recall@K
def recall_at_k(recommended, relevant, k=10):
    recommended = recommended[:k]
    hits = len(set(recommended) & set(relevant))
    if len(relevant) == 0:
        return 0
    return hits / len(relevant)


# Evaluation loop
for user in users:
    user_data = ratings[ratings['userId'] == user]

    if len(user_data) < 5:
        continue

    # Relevant movies = movies the user liked
    relevant = user_data[user_data['rating'] >= RELEVANCE_THRESHOLD]['movieId'].tolist()

    if len(relevant) < 3:
        continue

    # ---------- Content-Based ----------
    try:
        liked_movies = relevant[:3]
        rec_ids = []

        for movie_id in liked_movies:
            title = movies.loc[movies['movieId'] == movie_id, 'title'].values
            if len(title) == 0:
```

```
                continue

            recs = recommend_movies(title[0])  # this returns titles
            ids = movies[movies['title'].isin(recs)]['movieId'].tolist()
            rec_ids.extend(ids)

        rec_ids = list(set(rec_ids))[:K]

        precision_content.append(precision_at_k(rec_ids, relevant, K))
        recall_content.append(recall_at_k(rec_ids, relevant, K))

    except:
        pass

    # ---------- Collaborative Filtering ----------
    try:
        recs = recommend_movies_user(user)   # returns movieId + title
        rec_ids = recs['movieId'].tolist()

        precision_cf.append(precision_at_k(rec_ids, relevant, K))
        recall_cf.append(recall_at_k(rec_ids, relevant, K))

    except:
        pass

    # ---------- Matrix Factorization ----------
    try:
        recs = recommend_movies_svd(user)    # returns movieId + title
        rec_ids = recs['movieId'].tolist()

        precision_svd.append(precision_at_k(rec_ids, relevant, K))
        recall_svd.append(recall_at_k(rec_ids, relevant, K))

    except:
        pass


# Compute averages
content_precision = np.mean(precision_content) if precision_content else 0
content_recall = np.mean(recall_content) if recall_content else 0

cf_precision = np.mean(precision_cf) if precision_cf else 0
cf_recall = np.mean(recall_cf) if recall_cf else 0

svd_precision = np.mean(precision_svd) if precision_svd else 0
svd_recall = np.mean(recall_svd) if recall_svd else 0


# ========================================
# RMSE
# ========================================

from sklearn.metrics import mean_squared_error

actual, predicted = [], []

for _, row in ratings.sample(5000).iterrows():
    user = row['userId']
    movie = row['movieId']

    if user in predicted_ratings.index and movie in predicted_ratings.index:
        pass

# SVD RMSE
actual, predicted = [], []

for _, row in ratings.sample(5000).iterrows():
    user = row['userId']
    movie = row['movieId']

    if user in predicted_ratings.index and movie in predicted_ratings.columns:
        actual.append(row['rating'])
        predicted.append(predicted_ratings.loc[user, movie])

rmse_svd = np.sqrt(mean_squared_error(actual, predicted))
```

```
# CF RMSE
def predict_rating_cf(user_id, movie_id):
    sim_scores = user_similarity_df.loc[user_id]
    movie_ratings = user_item_matrix[movie_id].fillna(0)

    weighted_sum = np.dot(sim_scores, movie_ratings)
    sim_sum = np.abs(sim_scores).sum()

    if sim_sum == 0:
        return 0

    return weighted_sum / sim_sum


actual_cf = []
predicted_cf = []

for _, row in ratings.sample(5000).iterrows():
    user = row['userId']
    movie = row['movieId']

    if movie in user_item_matrix.columns:
        pred = predict_rating_cf(user, movie)
        actual_cf.append(row['rating'])
        predicted_cf.append(pred)

rmse_cf = np.sqrt(mean_squared_error(actual_cf, predicted_cf))


# ========================================
# Comparison Table
# ========================================

comparison = pd.DataFrame({
    "Model": [
        "Content-Based",
        "Collaborative Filtering",
        "Matrix Factorization (SVD)"
    ],
    "RMSE": [
        np.nan,
        rmse_cf,
        rmse_svd
    ],
    "Precision@10": [
        content_precision,
        cf_precision,
        svd_precision
    ],
    "Recall@10": [
        content_recall,
        cf_recall,
        svd_recall
    ]
})

display(comparison)


# Precision / Recall plot
comparison.set_index("Model")[["Precision@10", "Recall@10"]].plot(kind="bar", figsize=(8, 5))
plt.title("Recommendation Model Comparison")
plt.ylabel("Score")
plt.tight_layout()
plt.show()


# RMSE plot
comparison.dropna(subset=["RMSE"]).set_index("Model")["RMSE"].plot(kind="bar", figsize=(6, 4))
plt.title("RMSE Comparison")
plt.ylabel("RMSE")
plt.tight_layout()
plt.show()
```

	Model	RMSE	Precision@10	Recall@10
0	Content-Based	NaN	0.094	0.014792
1	Collaborative Filtering	3.057282	0.480	0.115530
2	Matrix Factorization (SVD)	2.014194	0.664	0.163459





Next steps: Generate code with comparison New interactive sheet

Model Evaluation and Comparison Summary

Based on the evaluation metrics, here's a summary and comparison of the three recommendation models:

- **Matrix Factorization (SVD)**: This model demonstrates the best performance overall, achieving the lowest RMSE of **2.014** and the highest Precision@10 (**0.664**) and Recall@10 (**0.163**). This suggests it is the most accurate at predicting ratings and providing relevant recommendations among the evaluated models.
- **Collaborative Filtering (User-User)**: This model shows good performance, especially in terms of Precision@10 (**0.480**) and Recall@10 (**0.116**), but its RMSE of **3.057** indicates that its rating predictions are less accurate than SVD. It relies on user similarity to make recommendations.
- **Content-Based**: This model has a significantly lower Precision@10 (**0.094**) and Recall@10 (**0.015**), and its RMSE is not available in this comparison. This suggests that for the chosen evaluation strategy and dataset, the content-

based approach (using genres and tags) is less effective than the other two models in providing relevant

```
# import pickle

# model_data = {
#     "tfidf": tfidf,
#     "tfidf_matrix": tfidf_matrix,
#     "movies": movies_with_tags,
#     "indices": indices
# }

# with open("model.pkl", "wb") as f:
#     pickle.dump(model_data, f)

# print("Model saved successfully!")
```

```
# from google.colab import files
# files.download("model.pkl")
```